Task 1 – Understand the Problem and Identify the Data

Python Programming • Assignment 1 • Village Milk Co-operative Payment Calculator | Marks: 4 | Bloom's level: K4 – Analyse

Name: **Dhaarani D** Register No.: _____ GitHub: _____ (folder Task01)

1. Inputs and outputs

The clerk records one entry every time a farmer delivers milk (twice a day). The program must turn these raw readings into money, so every value is classed as an input (typed or read), a derived value (calculated) or an output (printed or paid).

Value	Kind	Type	Why this type
farmer ID (e.g. F001)	input	str	An identifier is never used in arithmetic; leading zeros and letters must be kept, which an int would lose.
shift (M / E)	input	str	A label with two values; only compared, never added.
litres	input	float / Decimal	Fractional (8.5 L); measured quantity.
fat %, SNF %	input	float / Decimal	Fractional (3.5, 8.7); must be compared with 3.5 and 8.5 exactly.
fat steps, SNF steps	derived	int	A count of 0.1 % steps is always a whole number (7, −4).
rate per litre	output	money (2 decimals)	Rupees and paise; has to match the bank to the paisa.
amount for one entry	output	money (2 decimals)	litres × rate, rounded to whole paise.
ten-day total	output	money (2 decimals)	Sum of the ten days' amounts for one farmer; this is what is transferred.
number of entries	derived	int	Counts entries (about 20 per farmer in ten days).

2. Values that never change during a run (named constants)

A constant is any value that is part of the co-operative's rules rather than part of one delivery. Naming them once is the direct answer to the volunteer's problem of rates being typed in several places.

Constant	Value	Reason it is a constant
BASE_RATE	30.00	Price of standard milk; changes only in a union revision, never during a run.
FAT_STEP_VALUE	0.40	Rs added/subtracted per 0.1 % of fat from the standard.
SNF_STEP_VALUE	0.30	Rs added/subtracted per 0.1 % of SNF from the standard.
STD_FAT, STD_SNF	3.5, 8.5	The quality at which the base rate applies.
STEP	0.1	Size of one step in per cent.
MIN_FAT, MAX_FAT	2.0, 7.0	Valid range of fat given in the problem.
PAYMENT_DAYS	10	Payment cycle (used when totalling).

3. Whole numbers, fractional numbers and text

- **Whole numbers (int):** number of steps, number of entries, number of days, and – if we choose integer paise – the money itself.
- **Fractional numbers:** litres, fat %, SNF %, the rates and the amounts. Money is fractional only because rupees have paise; this is why Task 2 studies float versus Decimal.
- **Text (str):** farmer ID, shift, and everything the clerk types, because input() always returns a string and conversion is the program's job (Task 5).

4. Rules every correct payment must satisfy

Rule	Justification
litres > 0 (and not absurdly large, e.g. ≤ 200 per entry)	A negative or zero quantity would give a negative or zero payment; a typing slip such as 1000 instead of 10.0 would be paid silently.
2.0 ≤ fat ≤ 7.0	Given in the problem; outside this range the milk is either adulterated or the analyser/clerk has made an error.
6.0 ≤ SNF ≤ 10.0 (assumption)	Not stated in the problem, but an SNF of 0 or 90 is clearly an entry error; the limits are named constants so the union can change them.
fat and SNF must be numbers	A text value ("eight", "8,5") cannot be used in the step formula.
rate > 0 and amount = round(litres × rate, 2)	The computed rate must never become zero or negative; the amount must be an exact number of paise.
farmer is paid the sum of already-rounded entry amounts	Matches how the bank statement is built (each credit is a whole number of paise), so totals agree to the paisa.

5. Assumptions

- Steps are counted by rounding to the nearest whole 0.1 % (readings are given to one decimal).
- Rounding of a half paisa is *half up* (Rs 1.505 becomes Rs 1.51), as a bank does.
- One entry = one farmer, one shift; ten days = about 20 entries per farmer.
- SNF limits (6.0–10.0) and the 200 L per-entry cap are my own sanity limits and are easy to change.

Alternative Concept and Complexity Analysis

Alternative: keep every entry in three separate names (litres1, fat1, snf1, litres2, …). For 300 farmers this means 900 separate assignments typed by hand, and a calculation such as `total = litres1*rate(fat1,snf1) + litres2*rate(fat2,snf2) + …` with 300 terms. A Python loop cannot visit variable names, so the code must be rewritten whenever the number of farmers changes. Even the best imitation of this idea – a dictionary with keys "litres1", "fat1" … – needs a string to be built for every access. I measured it:

Both designs (excerpt of storage_compare.py)

```
import random, sys, time

def make(n):
    rnd = random.Random(1)
    return [(round(rnd.uniform(2, 25), 1), rnd.randint(25, 65) / 10, rnd.randint(78, 92) / 10) for _ in range(n)]

def rate(fat, snf):
    return 30.0 + round((fat - 3.5) / 0.1) * 0.40 + round((snf - 8.5) / 0.1) * 0.30

def total_tuples(entries):                    # design B: one tuple per entry
    t = 0.0
    for litres, fat, snf in entries:
        t += litres * rate(fat, snf)
    return t

def total_named(ns, n):                       # design A: litres1, fat1, snf1 ... as separate names
    t = 0.0
    for i in range(1, n + 1):
        t += ns[f"litres{i}"] * rate(ns[f"fat{i}"], ns[f"snf{i}"])
    return t
```

Output

```
n=   300  tuples    0.12 ms  named    0.30 ms  container       21 KB vs       67 KB (names)  same total: True
n=  6000  tuples    2.39 ms  named    7.05 ms  container      427 KB vs     1264 KB (names)  same total: True
n=150000  tuples   59.67 ms  named  276.69 ms  container    10628 KB vs    37105 KB (names)  same total: True
lines of code to declare 300 farmers with separate names: 900 assignments (3 x 300)
```

	Design A: separate names	Design B: one tuple per entry in a list
Time to process n entries	O(n) but with a larger constant (string building + dict lookup, ≈ 4–5× slower here); code length is O(n) lines	O(n) with a loop of 3 lines; code length O(1)
Space	O(n) variables plus their name strings (≈ 3.5× more memory here)	O(n) – one small tuple per entry
Scaling 300 → 6,000 → 150,000	Source file grows with n; impossible to type	Same program; only the data grows

Conclusion: both are O(n) in running time, but design B is O(1) in program text and has a lower constant; design A’s program grows with the data, so it does not scale beyond a handful of farmers.

Evidence

The tables above give the inputs, outputs, constants, types, rules and assumptions, each with its reason; the measurement shows why the list/tuple design is chosen. Source: Task01/storage_compare.py.

Task 2 – Explore Values and Types in the Interpreter

Python Programming • Assignment 1 • Village Milk Co-operative Payment Calculator | Marks: 4 | Bloom's level: K4 – Analyse

Name: **Dhaarani D** Register No.: _____ GitHub: _____ (folder Task02)

1. Interactive session

Python 3.12 interactive mode

```
>>> type(10)
<class 'int'>
>>> type(10.0)
<class 'float'>
>>> type("10")
<class 'str'>
>>> type(True)
<class 'bool'>
>>> type([10, 8.5])
<class 'list'>
>>> 7 / 2
3.5
>>> 7 // 2
3
>>> 7 % 2
1
>>> -7 // 2
-4
>>> -7 % 2
1
>>> 0.1 + 0.2
0.30000000000000004
>>> 0.1 + 0.2 == 0.3
False
>>> isinstance(True, int)
True
>>> "10" + "10"
'1010'
>>> 10 + float("10")
20.0
>>> from decimal import Decimal
>>> Decimal("0.1") + Decimal("0.2")
Decimal('0.3')
>>> Decimal(0.1)
Decimal('0.1000000000000000055511151231257827021181583404541015625')
>>> round(0.1 + 0.2, 2) == 0.3
True
>>> round(2.675, 2)
2.67
>>> round(0.5), round(1.5), round(2.5)
(0, 2, 2)
>>> int(19.99 * 100)
1998
>>> round(19.99 * 100)
1999
```

2. Explanation of each result

Expression	Result	Explanation
type(10), type(10.0), type("10"), type(True), type([10, 8.5])	int, float, str, bool, list	The value's form decides the type: 10.0 is a float because of the decimal point; quotes make a str; a list holds several values of any type.
7 / 2	3.5	**/** is true division and always returns a float.
7 // 2	3	**//** is floor division: the quotient rounded down.
7 % 2	1	Remainder, so that 7 = 2·3 + 1.
-7 // 2	-4	*Surprise:* floor division rounds toward minus infinity, not toward zero; −3.5 floors to −4. Then -7 % 2 is 1 so that 2·(−4) + 1 = −7 holds.

Expression	Result	Explanation
0.1 + 0.2	0.30000000000000004	*Surprise:* 0.1 and 0.2 have no exact binary form; the stored values are slightly off and the sum is a tiny bit more than 0.3.
0.1 + 0.2 == 0.3	False	The computer compares the stored binary numbers, not the decimals we typed. Decimal(0.1) shows the real stored value 0.1000000000000000055511...
isinstance(True, int)	True	*Surprise:* bool is a subclass of int (True is 1), so True + True is 2.
"10" + "10" vs 10 + float("10")	'1010' and 20.0	+ joins strings but adds numbers; this is why input text must be converted.
round(2.675, 2)	2.67	*Surprise:* 2.675 is stored as 2.67499999..., so it rounds down.
round(0.5), round(1.5), round(2.5)	0, 2, 2	*Surprise:* round() uses banker's rounding (ties go to the even number), not "half up".
int(19.99*100) vs round(19.99*100)	1998 vs 1999	19.99×100 is 1998.9999999999998; int() cuts it off, round() gives the intended 1999 paise.

3. Why money must not be added as float rupees

Float stores binary approximations, so each addition adds a tiny error and the errors accumulate; a bank total is exact to the paisa and the two will disagree. The alternatives are: **integer paise** (Rs 33.40 = 3340), **round()** after every operation, or the **decimal** module which stores decimal digits exactly.

4. Complete the Code – TODO 1

milk_money.py (TODO 1 replaced)

```
from decimal import Decimal
import sys, timeit

# 1,000 payments of Rs 33.40 and three small adjustments of Rs 0.10
payments = ["33.40"] * 1000 + ["0.10"] * 3


def total_float(items):
    total = 0.0
    for amount in items:
        total += float(amount)
    return total


def total_decimal(items):
    # TODO 1 completed: exact addition using Decimal built from each string
    total = Decimal("0")
    for amount in items:
        total += Decimal(amount)
    return total


print("float  :", total_float(payments))
print("Decimal:", total_decimal(payments))
print(0.1 + 0.2 == 0.3, Decimal("0.1") + Decimal("0.2") == Decimal("0.3"))
```

Output (identical to the expected output)

```
float  : 33400.30000000061
Decimal: 33400.30
False True
```

5. Change Decimal(amount) to Decimal(float(amount))

Output

```
Decimal(float(amount)) : 33400.29999999999857893118278
Decimal(float('33.40')): 33.399999999999985789145284797996282577514648437 5
```

The result becomes 33400.299999999998578931182 78 instead of 33400.30. `float("33.40")` is already the inexact binary number 33.39999999999999857...; Decimal then copies that error exactly. A Decimal is only exact when it is built from the **string** the clerk typed, never from a float.

Alternative Concept and Complexity Analysis

(a) Integer paise: convert each amount with `round(float(a)*100)`, add as int, divide by 100 only when printing. **(b) float + round():** `t = round(t + float(a), 2)` after every addition.

Measured (alternatives.py)

```
integer paise          : 3340030 paise  -> 33400.3 rupees
float + round()        : 33400.3
exact Decimal          : 33400.30

memory per value (sys.getsizeof):  int 28  float 24  Decimal 104
time per addition float (plain)       103 ns
time per addition integer paise       245 ns
time per addition float + round()     457 ns
time per addition Decimal             254 ns
```

Method	Time / addition	Memory / value	Exact bank total for n payments?
float (plain)	≈ 100 ns	24 bytes	No – 33400.30000000061 (error grows with n)
float + round()	≈ 460 ns	24 bytes	Gave 33400.3 here, but not guaranteed: round() on a float tie can go the wrong way (Task 10 shows 7,324 wrong paise)
integer paise	≈ 250 ns	28 bytes	Yes – integers are exact; needs care when converting from text
Decimal (from str)	≈ 250 ns	104 bytes	Yes – exact decimal arithmetic and explicit rounding modes

All are O(1) per addition and O(n) for n payments; the difference is only the constant factor and correctness. Decimal costs about 4× the memory of a float but is the only one that is both exact and readable.

Evidence

Annotated session with types, division operators and the 0.1 + 0.2 issue, resolved with Decimal built from strings. Source: Task02/milk_money.py, Task02/alternatives.py.

Task 3 – Choose Variables and Name the Constants

Python Programming • Assignment 1 • Village Milk Co-operative Payment Calculator | Marks: 3 | Bloom's level: K3 – Apply

Name: **Dhaarani D** Register No.: _____ GitHub: _____ (folder Task03)

1. The named constants (single place for every rate)

constants.py

```
# Rate card: change a rate HERE and nowhere else
BASE_RATE = 30.00        # Rs per litre at 3.5 % fat and 8.5 % SNF
FAT_STEP_VALUE = 0.40    # Rs per 0.1 % fat
SNF_STEP_VALUE = 0.30    # Rs per 0.1 % SNF
STD_FAT = 3.5
STD_SNF = 8.5
STEP = 0.1
MIN_FAT = 2.0
MAX_FAT = 7.0
PAISA = 0.01

def rate_per_litre(fat, snf):
    return round(BASE_RATE + round((fat - STD_FAT) / STEP) * FAT_STEP_VALUE
                 + round((snf - STD_SNF) / STEP) * SNF_STEP_VALUE, 2)
```

A rate revision now means editing **one line**; every function reads the name, so a revision can never be applied "partly" as it was in the volunteer's program.

2. Naming rules and conventions

- **Rules (enforced by Python):** a name may contain letters, digits and underscore; it cannot start with a digit (`2value = 5` is a SyntaxError); it cannot be a keyword (`if, for, class`); names are case-sensitive, so `fat_pct` and `Fat_pct` are two different variables.
- **Conventions (PEP 8):** variables and functions in lower_case_with_underscores (`fat_steps`, `rate_per_litre`); constants in UPPER_CASE (BASE_RATE); names must say what the value means (`fat_pct`, not `x`); single letters only for tiny loop counters.

3. Why Python has no true constants

Python has no `const` keyword. Any name can be assigned again, as the session below shows (BASE_RATE became 99 without any error). The language relies on the UPPER_CASE *convention* and on programmer discipline, code review and tests; for a stricter guarantee one can place constants in a separate module (Task 7) and only read them.

4. Dynamic typing, swapping and a type error – interactive session

Python 3.12 interactive mode

```
>>> x = 10
>>> type(x)
<class 'int'>
>>> x = "ten"
>>> type(x)
<class 'str'>
>>> x = 8.5
>>> type(x), x
(<class 'float'>, 8.5)
>>> a, b = 3.5, 8.5
>>> a, b = b, a
>>> a, b
(8.5, 3.5)
>>> "10" + 5
Traceback (most recent call last):
  File "<input>", line 1, in <module>
TypeError: can only concatenate str (not "int") to str
>>> BASE_RATE = 30.00
>>> BASE_RATE = 99
>>> BASE_RATE
99
```

```
>>> 2value = 5
  File "<input>", line 1
    2value = 5
    ^
SyntaxError: invalid decimal literal
>>> fat_pct = 4.2; Fat_pct = 3.1
>>> fat_pct, Fat_pct
(4.2, 3.1)
```

- **Dynamic typing:** the *name* x has no type; the *value* does. x was an int, then a str, then a float with no declaration. The risk is that `"10" + 5` fails only when that line runs (TypeError), which is why input must be converted and tested.
- **Multiple assignment and swap:** `a, b = b, a` builds the pair (b, a) first and then assigns, so no temporary variable is needed; a and b changed from (3.5, 8.5) to (8.5, 3.5).

Alternative Concept and Complexity Analysis

Alternative 1 – magic numbers: `30.0 + round((fat-3.5)/0.1)*0.40 + …` typed inside every expression. **Alternative 2 – dictionary:** `RATE_CARD = {"base": 30.0, "fat": 0.40, "snf": 0.30}` read as `RATE_CARD["fat"]`.

Measured (compare_styles.py)

```
magic numbers       734 ns per call
dictionary         1029 ns per call
named constants     752 ns per call
0.40 typed in 1 place per function; a program with k = 3 copies of the formula needs 3 edits
```

	Magic numbers	Dictionary RATE_CARD	Named constants
Places to edit for one revision	k (every copy of the formula; k = 3 in the volunteer's program) – easy to miss one	1	1
Time per rate calculation	≈ 734 ns	≈ 1029 ns (extra hash look-up per value)	≈ 752 ns
Memory	none extra	one small dict (≈ 200 bytes)	three names (≈ 24 bytes each)
Readability / risk	unreadable; numbers have no meaning	good; but a misspelt key fails only at run time (KeyError)	good; a misspelt name is caught on the first run (NameError)

All three are O(1) in time and space for one rate calculation, so running time and memory do **not** meaningfully change (the dictionary is about 40 % slower per call, only 0.3 µs). What changes is the **maintenance cost**: k edits and k chances of error versus one. The named constants are chosen: as fast as magic numbers and as maintainable as the dictionary.

Evidence

One well-named set of constants (constants.py) and an explanation of naming, constants by convention and dynamic typing, backed by an interpreter session. Source: Task03/constants.py, compare_styles.py.

Task 4 – Write Correct Expressions

Python Programming • Assignment 1 • Village Milk Co-operative Payment Calculator | Marks: 4 | Bloom's level: K4 – Analyse

Name: **Dhaarani D** Register No.: _____ GitHub: _____ (folder Task04)

1. The four expressions

Expressions used in the program

```
fat_steps = round((fat - STD_FAT) / STEP)          # whole 0.1 % steps from 3.5 (can be negative)
snf_steps = round((snf - STD_SNF) / STEP)          # whole 0.1 % steps from 8.5
rate      = round(BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE, 2)
amount    = round(litres * rate, 2)
valid_fat = MIN_FAT <= fat <= MAX_FAT              # comparison chaining
```

Worked example: litres 8.5, fat 4.2, SNF 8.7. fat_steps = round(7.000000000000002) = 7, snf_steps = round(1.999999999999993) = 2, rate = 30.00 + 7×0.40 + 2×0.30 = 33.40, amount = 8.5 × 33.40 = 283.90.

2. Precedence and associativity

Rule	Example	Result and reason
** binds tighter than unary minus	-2 ** 2	-4, because it is -(2**2); use (-2) ** 2 to get 4
** is right-associative	2 ** 3 ** 2	512 = 2 ** (3 ** 2), not (2**3)**2 = 64
unary minus on the right side of **	2 ** -1	0.5; the sign belongs to the exponent
* / // % are equal and left-associative	10 / 2 * 5	25.0 (left to right), not 1.0
+ and - are left-associative	5 - 3 - 1	1, i.e. (5-3)-1
comparison binds tighter than not, not tighter than and, and tighter than or	True or False and False	True: and is done first, then or

In the step formula the brackets `(fat - STD_FAT) / STEP` are essential: without them, `fat - STD_FAT / STEP` would divide only the constant.

3. Interpreter evidence (surprising results)

Python 3.12 interactive mode

```
>>> (4.2 - 3.5) / 0.1
7.000000000000002
>>> (3.1 - 3.5) / 0.1
-3.999999999999999
>>> (4.1 - 3.5) / 0.1
5.9999999999999964
>>> round((4.1 - 3.5) / 0.1)
6
>>> int((4.1 - 3.5) / 0.1)
5
>>> import math
>>> -2 ** 2
-4
>>> (-2) ** 2
4
>>> 2 ** 3 ** 2
512
>>> 2 ** -1
0.5
>>> 5 - 3 - 1
1
>>> 10 / 2 * 5
25.0
>>> int(-3.5), round(-3.5), math.floor(-3.5)
(-3, -4, -4)
>>> fat = 4.2
```

```
>>> 2.0 <= fat <= 7.0
True
>>> 2.0 <= fat and fat <= 7.0
True
>>> not 2.0 <= fat <= 3.0
True
>>> True or False and False
True
```

4. Three (and more) unexpected results explained

Expression	Result	Explanation
(4.2 - 3.5) / 0.1	7.000000000000002	None of 4.2, 3.5 or 0.1 is stored exactly, so the quotient is a hair above 7 (the brief quotes ...001; the last digits depend on the exact numbers but the point is the same). round() gives the intended 7.
(4.1 - 3.5) / 0.1	5.999999999999964	**Worse:** slightly *below* 6. int() would give 5 and a wrong rate; round() gives 6.
-2 ** 2	-4	Precedence: ** before unary minus.
int(-3.5), round(-3.5), math.floor(-3.5)	-3, -4, -4	int() truncates toward zero; floor goes to the lower integer; round(-3.5) goes to the even neighbour (-4).
round(2.5)	2	Banker's rounding (shown in Task 2).

5. round(), int() and math.floor() for negative values

For positive numbers int() and floor() agree (both cut off the fraction). For negatives they differ: int(−3.9) = −3 (toward zero), math.floor(−3.9) = −4 (toward minus infinity), round(−3.9) = −4 (nearest). For a step count we want the *nearest* whole step, which is exactly round(), because the only error is the tiny representation error of about 10^{-15}.

6. Comparison chaining and Boolean operators

`2.0 <= fat <= 7.0` means `2.0 <= fat and fat <= 7.0` with fat evaluated once; both give True for 4.2 (session above). It is how the range rule of Task 1 is written. `not` is the highest of the Boolean operators, then and, then `or`. Note that `2.0 <= nan <= 7.0` is False, so a range test also rejects a "nan" reading.

Alternative Concept and Complexity Analysis

Alternative: compute the steps with int() (truncation) or math.floor() instead of round().

steps.py (excerpt)

```
import math

STD_FAT, STD_SNF, STEP = 3.5, 8.5, 0.1
BASE, FAT_V, SNF_V = 30.00, 0.40, 0.30

def rate(fat, snf, how):
    fs = how((fat - STD_FAT) / STEP)
    ss = how((snf - STD_SNF) / STEP)
    return round(BASE + fs * FAT_V + ss * SNF_V, 2), fs
```

Measured (steps.py; SNF fixed at 8.5)

```
method        fat             raw steps  steps  rate (snf 8.5)
round()       4.2     7.000000000000002      7           32.80
int()         4.2     7.000000000000002      7           32.80
math.floor    4.2     7.000000000000002      7           32.80
round()       3.1    -3.999999999999999     -4           28.40
int()         3.1    -3.999999999999999     -3           28.80
math.floor    3.1    -3.999999999999999     -4           28.40
round()       4.1     5.999999999999996      6           32.40
int()         4.1     5.999999999999996      5           32.00
math.floor    4.1     5.999999999999996      5           32.00
```

Fat	round()	int()	math.floor()	Comment
4.2	7 steps → Rs 32.80	7 → 32.80	7 → 32.80	All agree because the raw value is just above 7.
3.1	−4 → Rs 28.40 (correct)	−3 → Rs 28.80 (Rs 0.40 too high)	−4 → Rs 28.40	int() truncates toward zero, so the farmer is overpaid; floor is right *by luck* (raw −3.999…).
4.1 (extra)	6 → Rs 32.40 (correct)	5 → Rs 32.00	5 → Rs 32.00	Raw 5.99…: both int and floor lose a whole step (Rs 0.40/L).

Cost: each method is a single operation, O(1) time and O(1) space; there is no performance reason to prefer one. The choice is purely about correctness, and only round() is correct for both signs and for representation errors of either direction.

Evidence

Correct expressions with several demonstrated surprises, each explained. Source: Task04/steps.py and the interactive session.

Task 5 – Read Input and Convert Types

Python Programming • Assignment 1 • Village Milk Co-operative Payment Calculator | Marks: 3 | Bloom's level: K3 – Apply

Name: **Dhaarani D** Register No.: _____ GitHub: _____ (folder Task05)

1. input() always returns a string

Whatever the clerk types, `input()` returns a **str**: typing 8.5 gives "8.5", not the number 8.5. Arithmetic on it would fail or, worse, give nonsense ("8.5" * 2 is "8.58.5"). The program must convert explicitly: `int(text)` for whole numbers (e.g. number of days) and `float(text)` or `Decimal(text)` for litres, fat and SNF.

2. What happens with "8,5" or "eight"

`float("8,5")` and `float("eight")` both stop the program with `ValueError: could not convert string to float`. The comma is a decimal separator in some regions, and the clerk may well type it, so the program must reject it with a clear message rather than crash or guess.

3. Validating before converting (string methods)

read_entry.py

```
def read_float(prompt, low, high):
    """Keeps asking until the clerk types a number between low and high."""
    while True:
        text = input(prompt).strip()
        if not text.replace(".", "", 1).isdigit():
            print("  Please type digits with an optional decimal point, e.g. 8.5")
            continue
        value = float(text)
        if not (low <= value <= high):
            print(f"  Value must be between {low} and {high}")
            continue
        return value

litres = read_float("Litres  : ", 0.1, 200)
fat = read_float("Fat %   : ", 2.0, 7.0)
snf = read_float("SNF %   : ", 6.0, 10.0)
print(f"Accepted: {litres:8.1f} L  fat {fat:4.1f}  snf {snf:4.1f}")
```

Run with the inputs: eight, 8,5, 10, 9.5, 4.2, 8.7

```
Litres  : eight
  Please type digits with an optional decimal point, e.g. 8.5
Litres  : 8,5
  Please type digits with an optional decimal point, e.g. 8.5
Litres  : 10
Fat %   : 9.5
  Value must be between 2.0 and 7.0
Fat %   : 4.2
SNF %   : 8.7
Accepted:     10.0 L  fat  4.2  snf  8.7
```

`text.replace(".", "", 1).isdigit()` removes at most one decimal point and then requires every remaining character to be a digit. Two checks are made separately: *is it a number?* and *is it in the valid range?*, so each error message is specific.

4. Formatted output with f-strings

width, alignment (<, >) and precision (.2f)

```
print(f"{'ID':<6}{'Litres':>8}{'Fat':>6}{'SNF':>6}{'Rate':>8}{'Amount':>10}")
print(f"{fid:<6}{l:8.1f}{f:6.1f}{s:6.1f}{r:8.2f}{a:10.2f}")
```

Aligned table

```
ID      Litres   Fat   SNF    Rate    Amount
F001      10.0   3.5   8.5   30.00    300.00
F002       8.5   4.2   8.7   33.40    283.90
F003      12.0   3.1   8.3   27.80    333.60
```

`{amount:10.2f}` means: a float in a field 10 characters wide with exactly 2 decimals; < left-aligns text and > right-aligns numbers so decimal points line up.

Alternative Concept and Complexity Analysis

Alternative: skip the pre-check and use `try: float(text) except ValueError:`.

entry_input.py (excerpt)

```
import timeit

def is_number(text):
    """String-method check: optional spaces, digits with at most one decimal point."""
    t = text.strip()
    return t.replace(".", "", 1).isdigit()

def to_float_checked(text):
    if is_number(text):
        return float(text)
    return None

def to_float_try(text):
    try:
        return float(text)
    except ValueError:
        return None
```

Measured behaviour and time (entry_input.py)

```
input        is_number + float    try/except float
'8.5'                      8.5                 8.5
'8,5'                     None                None
'eight'                   None                None
' 8.5 '                    8.5                 8.5
'-1'                      None                -1.0
'1e3'                     None              1000.0
''                        None                None
'nan'                     None                 nan
'inf'                     None                 inf
timing for '8.5'    check-first    301 ns   try/except    128 ns
timing for 'eight'  check-first    165 ns   try/except    813 ns
```

Input	Check first (strip + isdigit)	try / except float()
"8.5"	accepted (8.5)	accepted (8.5)
"8,5"	rejected – clearly because of the comma	rejected via ValueError
" 8.5 "	accepted after strip()	accepted: float() ignores surrounding spaces
"eight"	rejected	rejected via ValueError
"-1", "1e3"	rejected (not plain digits)	**accepted** as −1.0 and 1000.0 – valid floats but invalid litres, so a range check is still needed
"nan", "inf"	rejected	**accepted** as float values; only a range check (which is False for nan) stops them

Time: both are O(1) per entry for a short string (O(L) for length L). For a valid entry the pre-check costs about 300 ns against 130 ns for try/except; for an invalid entry the pre-check costs about 165 ns against 810 ns because raising an exception is expensive. Since entries are typed by a person (seconds each), the difference is irrelevant. **Clarity:** try/except is shorter and handles every format float() understands, but it also accepts "nan"/"inf"; the string check is stricter. The robust answer is to use try/except *and* a range check, or the string check followed by the range check, as in the program.

Evidence

Correct conversion, rejection of invalid input and neatly aligned output, shown above. Source: Task05/read_entry.py, entry_input.py.

Task 6 – Organise the Logic into Functions

Python Programming • Assignment 1 • Village Milk Co-operative Payment Calculator | Marks: 4 | Bloom's level: K3 – Apply

Name: **Dhaarani D** Register No.: _____ GitHub: _____ (folder Task06)

1. Functions, parameters and return values

Function	Parameters	Returns	Purpose
rate_per_litre(fat, snf)	fat %, SNF % (float)	rate in Rs (float, 2 dp)	applies the step rules to BASE_RATE
payout(litres, fat, snf)	litres, fat, SNF	amount in Rs (float, 2 dp)	litres × rate, rounded to paise
ten_day_total(entries)	list of (litres, fat, snf) tuples	total in Rs	adds the payout of every entry

functions.py

```
BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30
STD_FAT, STD_SNF, STEP = 3.5, 8.5, 0.1


def rate_per_litre(fat, snf):
    """Return the price per litre (float, 2 decimals) for the given fat and SNF %."""
    fat_steps = round((fat - STD_FAT) / STEP)
    snf_steps = round((snf - STD_SNF) / STEP)
    return round(BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE, 2)


def payout(litres, fat, snf):
    """Return the amount payable (float, 2 decimals) for one entry."""
    return round(litres * rate_per_litre(fat, snf), 2)


def ten_day_total(entries):
    """Return the total payable for a list of (litres, fat, snf) tuples."""
    total = 0.0
    for litres, fat, snf in entries:
        total += payout(litres, fat, snf)
    return round(total, 2)


def show_payout(litres, fat, snf):
    """PRINT a payout (returns None) - cannot be reused in further calculation."""
    print(f"pay {payout(litres, fat, snf):.2f}")


def bad_change_constant():
    BASE_RATE = 99.0            # creates a NEW local name; the global is untouched
    return BASE_RATE


if __name__ == "__main__":
    entries = [(10, 3.5, 8.5), (8.5, 4.2, 8.7), (12, 3.1, 8.3)]
    print("ten_day_total :", ten_day_total(entries))
    r = show_payout(8.5, 4.2, 8.7)
    print("show_payout returned:", r)
    print("payout returned     :", payout(8.5, 4.2, 8.7), "(a value we can add up)")
    print("local BASE_RATE     :", bad_change_constant(), " global BASE_RATE:", BASE_RATE)
    print("rate_per_litre doc  :", rate_per_litre.__doc__)
```

Output

```
ten_day_total : 917.5
pay 283.90
show_payout returned: None
payout returned     : 283.9 (a value we can add up)
local BASE_RATE     : 99.0  global BASE_RATE: 30.0
```

```
rate_per_litre doc  : Return the price per litre (float, 2 decimals) for the given fat and SNF %.
```

2. Returning a value versus printing it

`payout()` **returns** 283.9, a value the caller can store, add up, test with assert or print in any format. `show_payout()` only **prints**; it returns None, so its result cannot be added to a total or checked in a test. Calculation functions should return; only the outermost part of the program should print.

3. Local and global names

Names created inside a function (fat_steps, snf_steps) are **local**: they exist only during the call, so different functions cannot disturb each other. The rate constants are **global** and a function may *read* them freely. If a function assigns to the same name (`BASE_RATE = 99.0` in bad_change_constant) Python creates a *new local* variable and the global stays 30.0, exactly as the output shows (local 99.0, global 30.0). Changing a global from inside a function needs the `global` keyword, which should be avoided because it makes the rate depend on which function ran last.

4. Docstrings

Every function starts with a one-line docstring stating what it returns; `help(payout)` or `payout.__doc__` shows it (last line of the output above).

Alternative Concept and Complexity Analysis

Alternative 1 – one long script: the rate formula pasted inside the loop. **Alternative 2 – lambda expressions:** `rate_l = lambda fat, snf: ...`.

Measured (overhead.py, 150,000 entries, best of 5)

```
functions        177.8 ms for 150000 entries   total 68868741.06
inline script    161.4 ms for 150000 entries   total 68868741.06
lambda           179.4 ms for 150000 entries   total 68868741.06
```

	Functions (chosen)	Inline script	Lambda
Repeated code	none – formula written once	formula copied into every place that needs it; a rate change must be repeated in each copy	none, but one-line only; no docstring, no statements
Time, 150,000 entries	≈ 178 ms	≈ 161 ms	≈ 179 ms
Time per entry	O(1)	O(1)	O(1)
Space	O(1) extra (one stack frame per call)	O(1)	O(1)
Testable on its own	yes	no	awkward (anonymous)

Calling two functions per entry adds about 110 ns per entry (≈ 10 %), i.e. 16 ms for the whole district union (150,000 entries); this is not noticeable. Lambda and def are equally fast because a lambda *is* a function. Functions are therefore chosen: no measurable cost, but no repeated code and each part can be tested separately.

Evidence

Well-named functions with parameters, return values and docstrings, plus an explanation of scope shown by running code. Source: Task06/functions.py, overhead.py.

Task 7 – Build a Module

Python Programming • Assignment 1 • Village Milk Co-operative Payment Calculator | Marks: 4 | Bloom's level: K4 – Analyse

Name: **Dhaarani D** Register No.: _____ GitHub: _____ (folder Task07)

1. Complete the Code – TODO 1 and TODO 2 (milkrate.py)

milkrate.py

```
"""Rate card of the village milk co-operative (used as a module)."""
BASE_RATE = 30.00       # Rs per litre for milk with 3.5 % fat and 8.5 % SNF
FAT_STEP_VALUE = 0.40   # Rs per litre for every 0.1 % of fat above or below 3.5
SNF_STEP_VALUE = 0.30   # Rs per litre for every 0.1 % of SNF above or below 8.5
STD_FAT = 3.5
STD_SNF = 8.5
STEP = 0.1


def rate_per_litre(fat, snf):
    """Returns the price per litre, rounded to 2 decimals."""
    fat_steps = round((fat - STD_FAT) / STEP)
    snf_steps = round((snf - STD_SNF) / STEP)
    return round(BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE, 2)


def payout(litres, fat, snf):
    """Returns the amount payable to the farmer, rounded to 2 decimals."""
    return round(litres * rate_per_litre(fat, snf), 2)


if __name__ == "__main__":
    for litres, fat, snf in [(10, 3.5, 8.5), (8.5, 4.2, 8.7), (12, 3.1, 8.3)]:
        rate, pay = rate_per_litre(fat, snf), payout(litres, fat, snf)
        print(f"{litres:5} L  fat {fat}  snf {snf}  rate {rate:6.2f}  pay {pay:8.2f}")
```

python milkrate.py (identical to the expected output)

```
   10 L  fat 3.5  snf 8.5  rate  30.00  pay   300.00
  8.5 L  fat 4.2  snf 8.7  rate  33.40  pay   283.90
   12 L  fat 3.1  snf 8.3  rate  27.80  pay   333.60
```

2. Using the module from a separate script

use_milkrate.py

```
import sys
import milkrate
from milkrate import payout
import milkrate as mr

print("module name   :", milkrate.__name__)
print("payout(8.5, 4.2, 8.7) =", payout(8.5, 4.2, 8.7))
print("mr.rate_per_litre(3.1, 8.3) =", mr.rate_per_litre(3.1, 8.3))
print("loaded once   :", "milkrate" in sys.modules)
```

python use_milkrate.py – note that no table is printed

```
module name   : milkrate
payout(8.5, 4.2, 8.7) = 283.9
mr.rate_per_litre(3.1, 8.3) = 27.8
loaded once   : True
```

Why the table is not printed on import: when a file is run directly Python sets its __name__ to "__main__", so the block under `if __name__ == "__main__":` runs. When the same file is *imported*, __name__ is the module's file name, "milkrate" (printed above), the condition is False and the demonstration is skipped. The module therefore provides only definitions to its importer.

3. The name test demonstrated without the guard

A copy of the module without the guard, and a script importing it

```
"""Same module WITHOUT the __name__ test - used only to demonstrate the problem."""
def rate_per_litre(fat, snf):
    return round(30.00 + round((fat - 3.5) / 0.1) * 0.40 + round((snf - 8.5) / 0.1) * 0.30, 2)

for litres, fat, snf in [(10, 3.5, 8.5), (8.5, 4.2, 8.7)]:
    print("demo row:", litres, fat, snf, rate_per_litre(fat, snf))

# use_noguard.py:
import milkrate_noguard
print("rate =", milkrate_noguard.rate_per_litre(4.2, 8.7))
```

Output: the demo rows appear merely because the module was imported

```
demo row: 10 3.5 8.5 30.0
demo row: 8.5 4.2 8.7 33.4
rate = 33.4
```

4. Three ways to import

Statement	Name that becomes available	Call written as	Use when
import milkrate	milkrate	milkrate.payout(...)	Clear where a function comes from; no name clashes.
from milkrate import payout	payout (only)	payout(...)	Short code when only one or two names are needed; risk of clashing with another payout.
import milkrate as mr	mr	mr.payout(...)	A shorter alias for a long module name (as in import numpy as np).

5. Standard modules and where Python looks for a module

math gives floor, ceil, isclose; **decimal** gives exact decimal arithmetic (Task 2); **time**/**timeit** give timing (Task 10); **json** reads the rate card (Task 11). When `import milkrate` is executed Python searches, in order: modules already loaded (`sys.modules`), built-in modules, then each folder in `sys.path`: first the folder of the running script (shown as '' / the script folder), then the standard library folders and site-packages. That is why use_milkrate.py works only when milkrate.py is in the same folder (or its folder has been added to sys.path).

Alternative Concept and Complexity Analysis

Alternative: copy rate_per_litre() and payout() into every script that needs them.

Measured import cost

```
sys.path[0:2] = ['', '/usr/lib/python312.zip']
first import 152.4 us, later imports 114 ns each
```

	Module (import)	Copy into every script
Work when a rate changes	edit 1 file; every script sees the change on its next run	edit every copy (k scripts = k edits); a forgotten copy silently pays the old rate – the original volunteer's problem
One-time cost	first import ≈ 150 µs to find, compile (or load the cached .pyc) and run the file	none
Repeated cost	later `import` statements only look in sys.modules: ≈ 0.1 µs	none
Space	one module object in memory, shared by all importers: O(1)	k copies of the code: O(k) text and k copies of the function objects

The cost of importing is paid once per program run and is negligible beside the time to process even 300 entries; the maintenance saving is large. The module is therefore the better design.

Evidence

A working module imported by a separate script, with the __name__ test explained by demonstration. Source: Task07/milkrate.py, use_milkrate.py, milkrate_noguard.py, use_noguard.py.

Task 8 – Plan the Algorithm Before Coding

Python Programming • Assignment 1 • Village Milk Co-operative Payment Calculator | Marks: 4 | Bloom's level: K3 – Apply

Name: **Dhaarani D** Register No.: _____ GitHub: _____ (folder Task08)

1. Steps of algorithmic problem solving

Step	Applied to the milk payment
1. Understand	Input: the entries of one farmer for ten days. Output: amount payable. Rules: Task 1.
2. Plan	Pseudocode and flowchart below; hand trace on three entries before any Python is written.
3. Write	Translate the pseudocode line by line into functions (Tasks 6, 9).
4. Test	Compare program output with the hand trace and the five-entry hand total (Task 9).

2. Pseudocode – one payment and the ten-day total for one farmer

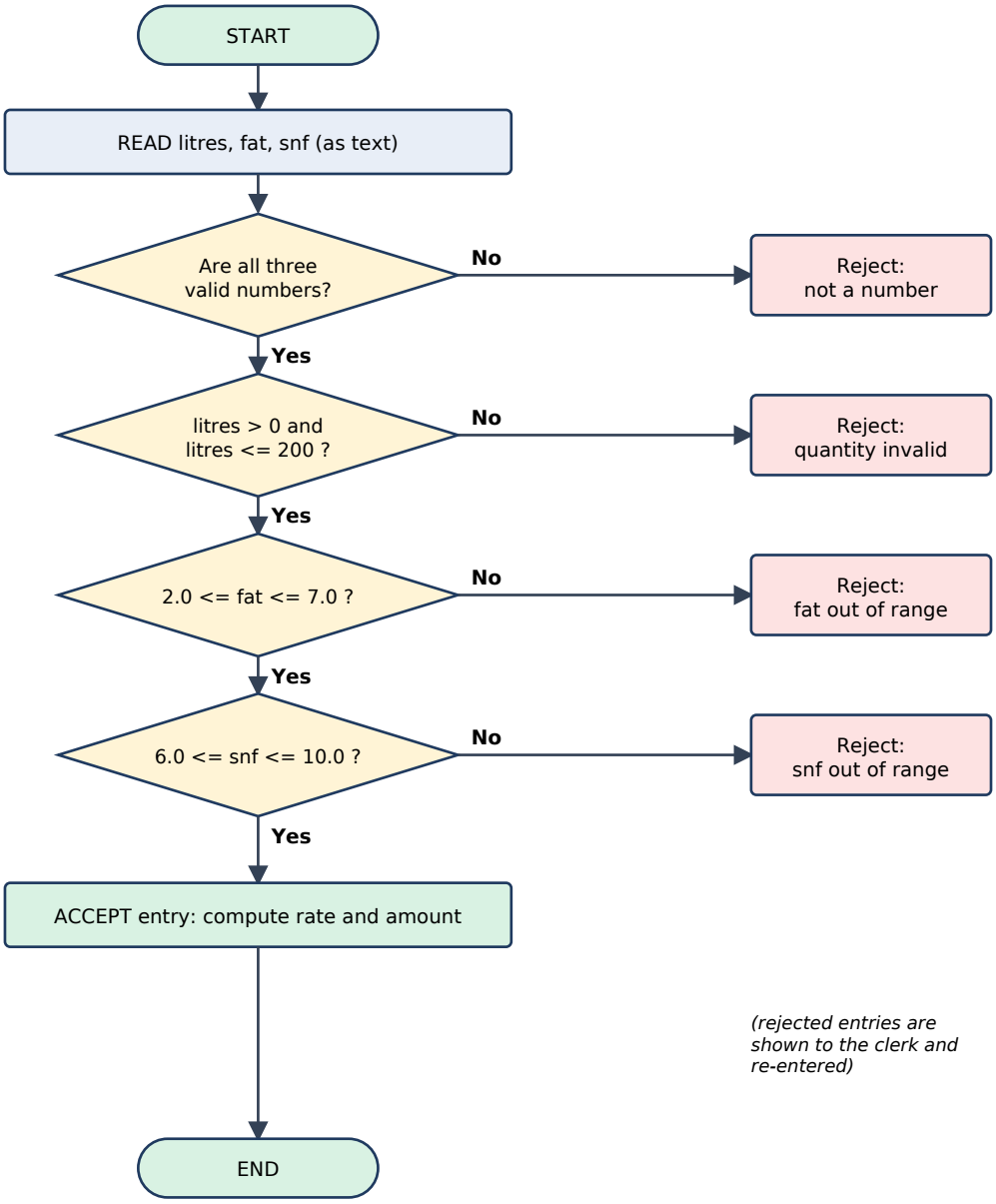
Pseudocode

```
ALGORITHM TenDayPayment
INPUT   entries : list of (litres, fat, snf), one item per delivery in the ten days
OUTPUT  total   : amount payable to the farmer in rupees (2 decimals)
CONSTANT BASE_RATE = 30.00, FAT_STEP_VALUE = 0.40, SNF_STEP_VALUE = 0.30
CONSTANT STD_FAT = 3.5, STD_SNF = 8.5, STEP = 0.1, MIN_FAT = 2.0, MAX_FAT = 7.0

BEGIN
 1   SET total TO 0.00
 2   FOR EACH (litres, fat, snf) IN entries DO
 3       IF litres <= 0 OR fat < MIN_FAT OR fat > MAX_FAT THEN
 4           DISPLAY "Invalid entry"
 5           CONTINUE with the next entry
 6       END IF
 7       SET fatSteps TO ROUND((fat - STD_FAT) / STEP)
 8       SET snfSteps TO ROUND((snf - STD_SNF) / STEP)
 9       SET rate     TO ROUND(BASE_RATE + fatSteps * FAT_STEP_VALUE + snfSteps * SNF_STEP_VALUE, 2)
10       SET amount   TO ROUND(litres * rate, 2)
11       SET total    TO total + amount
12   END FOR
13   RETURN total
END
```

Note: the keywords (BEGIN/END, SET … TO, FOR EACH, IF … END IF, RETURN) and the numbered lines follow the usual structured-pseudocode conventions; if the SIMATS standard sheet prescribes different keywords or layout, only the wording of each line needs to change, not the logic.

3. Flowchart – validating one entry



4. Hand trace of the pseudocode for three entries

Entry	litres	fat	snf	fatSteps (l.7)	snfSteps (l.8)	rate (l.9)	amount (l.10)	total (l.11)
1	10	3.5	8.5	0	0	30.00	300.00	300.00
2	8.5	4.2	8.7	7	2	33.40	283.90	583.90
3	12	3.1	8.3	−4	−2	27.80	333.60	917.50

Line 9 for entry 3: 30.00 + (−4)(0.40) + (−2)(0.30) = 30.00 − 1.60 − 0.60 = 27.80; line 10: 12 × 27.80 = 333.60. The program output below agrees with the trace (the final total 917.50 equals ten_day_total in Task 6).

Program output (trace.py)

```
  i litres  fat  snf fat steps snf steps   rate   amount    total
  1     10  3.5  8.5         0         0  30.00   300.00   300.00
  2    8.5  4.2  8.7         7         2  33.40   283.90   583.90
  3     12  3.1  8.3        -4        -2  27.80   333.60   917.50
```

Alternative Concept and Complexity Analysis

Solution 1 – store every entry's amount in a list, add at the end: `amounts.append(payout(...))` then `sum(amounts)`. **Solution 2 – running total while reading** (the pseudocode above).

Measured with tracemalloc (two_ways.py); times include tracemalloc overhead

```
 n=   300 | list_then_add      3.9 ms  peak     34.1 KB  total 135524.20 | running_total      3.7 ms  peak      0.7 KB
 n=  6000 | list_then_add     75.9 ms  peak    661.6 KB  total 2754827.26 | running_total     72.6 ms  peak      0.7 KB
 n=150000 | list_then_add   2097.8 ms  peak  16488.0 KB  total 68772194.07 | running_total   1897.3 ms  peak      0.7 KB
```

	List then add	Running total
Time for n entries	O(n): one payout per entry + one pass to add	O(n): one payout per entry; addition is done during the same pass
Extra space	O(n): 16.5 MB peak at n = 150,000	O(1): about 0.7 KB at every n
Keeps individual amounts?	yes	no

Both are O(n) in time (the list version needs one extra pass to add the amounts, a small constant); the space differs: O(n) versus O(1). At district scale, running totals keep memory flat. **Storing the entries is still necessary** when the amounts are needed again: printing a per-entry statement for the farmer, sorting or finding the highest/lowest delivery, computing an average or median, auditing a payment, or correcting one wrongly entered value afterwards.

Evidence

Pseudocode, flowchart and hand trace that match the final program (Task 9 uses the same rules and constants).

Task 9 – Implement and Test the Calculator

Python Programming • Assignment 1 • Village Milk Co-operative Payment Calculator | Marks: 4 | Bloom's level: K3 – Apply

Name: **Dhaarani D** Register No.: _____ GitHub: _____ (folder Task09)

1. Working source code (milkcalc.py)

Design choices from Tasks 1–8: money and readings are **Decimal** built from strings; all rates are named constants at the top; steps use round-half-up; every entry is validated; all calculation functions return values and only the demonstration at the bottom prints.

milkcalc.py

```
"""Milk co-operative payment calculator (final design: Decimal money, validated input)."""
from decimal import Decimal, InvalidOperation, ROUND_HALF_UP

# ---- rate card: the ONLY place where rates are typed ----
BASE_RATE = Decimal("30.00")      # Rs per litre at standard quality
FAT_STEP_VALUE = Decimal("0.40")  # Rs per 0.1 % fat
SNF_STEP_VALUE = Decimal("0.30")  # Rs per 0.1 % SNF
STD_FAT, STD_SNF = Decimal("3.5"), Decimal("8.5")
STEP = Decimal("0.1")
MIN_FAT, MAX_FAT = Decimal("2.0"), Decimal("7.0")
MIN_SNF, MAX_SNF = Decimal("6.0"), Decimal("10.0")   # assumption
MAX_LITRES = Decimal("200")                          # assumption: per entry
PAISA = Decimal("0.01")


def to_money(value):
    """Rounds a Decimal to whole paise (half up, as banks do)."""
    return value.quantize(PAISA, rounding=ROUND_HALF_UP)


def parse_number(text, name):
    """Converts text to Decimal or raises ValueError with a clear message."""
    try:
        value = Decimal(text.strip())
    except InvalidOperation:
        raise ValueError(f"{name}: '{text}' is not a number") from None
    if not value.is_finite():
        raise ValueError(f"{name}: '{text}' is not a finite number")
    return value


def validate_entry(litres, fat, snf):
    """Checks the business rules; raises ValueError if any rule is broken."""
    if not (0 < litres <= MAX_LITRES):
        raise ValueError(f"litres must be above 0 and at most {MAX_LITRES}")
    if not (MIN_FAT <= fat <= MAX_FAT):
        raise ValueError(f"fat must be between {MIN_FAT} and {MAX_FAT}")
    if not (MIN_SNF <= snf <= MAX_SNF):
        raise ValueError(f"snf must be between {MIN_SNF} and {MAX_SNF}")


def rate_per_litre(fat, snf):
    """Price per litre in Decimal rupees (2 decimals)."""
    fat_steps = ((fat - STD_FAT) / STEP).to_integral_value(rounding=ROUND_HALF_UP)
    snf_steps = ((snf - STD_SNF) / STEP).to_integral_value(rounding=ROUND_HALF_UP)
    return to_money(BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE)


def payout(litres, fat, snf):
    """Amount payable for one entry, rounded to the paisa."""
    validate_entry(litres, fat, snf)
    return to_money(litres * rate_per_litre(fat, snf))


def ten_day_total(entries):
    """Total of payouts for a list of (litres, fat, snf) Decimal tuples."""
    total = Decimal("0.00")
    for litres, fat, snf in entries:
        total += payout(litres, fat, snf)
```

```
    return total


def read_entry(l_text, f_text, s_text):
    """Parses three strings and returns the payout (ValueError on bad input)."""
    litres = parse_number(l_text, "litres")
    fat = parse_number(f_text, "fat")
    snf = parse_number(s_text, "snf")
    return payout(litres, fat, snf)


if __name__ == "__main__":
    print("Farmer  Litres   Fat   SNF     Rate      Amount")
    for fid, (l, f, s) in {"F001": ("10", "3.5", "8.5"), "F002": ("8.5", "4.2", "8.7"),
                           "F003": ("12", "3.1", "8.3")}.items():
        L, F, S = Decimal(l), Decimal(f), Decimal(s)
        print(f"{fid}  {L:7} {F:5} {S:5} {rate_per_litre(F, S):8.2f} {payout(L, F, S):10.2f}")
```

python milkcalc.py

```
Farmer  Litres   Fat   SNF     Rate      Amount
F001       10   3.5   8.5    30.00     300.00
F002      8.5   4.2   8.7    33.40     283.90
F003       12   3.1   8.3    27.80     333.60
```

2. Test program (assert-based)

test_milkcalc.py

```
from decimal import Decimal as D
import milkcalc as mc

results = []

def check(label, expected, actual):
    ok = expected == actual
    results.append(ok)
    print(f"{label:<38}{str(expected):>10}{str(actual):>10}  {'PASS' if ok else 'FAIL'}")
    assert ok, label            # stops at the first failure

def verdict(l, f, s):
    try:
        mc.read_entry(l, f, s)
    except ValueError:
        return "rejected"
    return "accepted"

print(f"{'Test case':<38}{'Expected':>10}{'Actual':>10}  Result")
# --- normal entries (above and below the standard) ---
check("N1 fat+ snf+  8.5 L 4.2/8.7", D("283.90"), mc.payout(D("8.5"), D("4.2"), D("8.7")))
check("N2 fat- snf-  12 L 3.1/8.3",  D("333.60"), mc.payout(D("12"), D("3.1"), D("8.3")))
check("N3 fat+ snf-  10 L 4.0/8.2",  D("311.00"), mc.payout(D("10"), D("4.0"), D("8.2")))
check("N4 fat- snf+  5 L 3.0/8.8",   D("144.50"), mc.payout(D("5"), D("3.0"), D("8.8")))
# --- boundary entries ---
check("B1 standard   10 L 3.5/8.5",  D("300.00"), mc.payout(D("10"), D("3.5"), D("8.5")))
check("B2 lowest fat 10 L 2.0/8.5",  D("240.00"), mc.payout(D("10"), D("2.0"), D("8.5")))
check("B3 highest fat 10 L 7.0/8.5", D("440.00"), mc.payout(D("10"), D("7.0"), D("8.5")))
check("B4 smallest qty 0.1 L 3.5/8.5", D("3.00"), mc.payout(D("0.1"), D("3.5"), D("8.5")))
check("B5 half-paise 0.05 L 3.6/8.4", D("1.51"), mc.payout(D("0.05"), D("3.6"), D("8.4")))
# --- invalid entries (all must be rejected) ---
for label, args in [("I1 text litres 'eight'", ("eight", "3.5", "8.5")),
                    ("I2 negative litres '-5'", ("-5", "3.5", "8.5")),
                    ("I3 zero litres '0'", ("0", "3.5", "8.5")),
                    ("I4 fat above range 7.1", ("10", "7.1", "8.5")),
                    ("I5 fat below range 1.9", ("10", "1.9", "8.5")),
                    ("I6 comma decimal '8,5'", ("10", "3.5", "8,5")),
                    ("I7 'nan' as fat", ("10", "nan", "8.5")),
                    ("I8 snf above range 11", ("10", "3.5", "11"))]:
    check(label, "rejected", verdict(*args))
# --- hand-calculated set of 5 entries ---
five = [(D("10"), D("3.5"), D("8.5")), (D("8.5"), D("4.2"), D("8.7")), (D("12"), D("3.1"), D("8.3")),
        (D("10"), D("4.0"), D("8.2")), (D("5"), D("3.0"), D("8.8"))]
check("H1 five-entry total (by hand)", D("1373.00"), mc.ten_day_total(five))
print(f"\n{sum(results)} of {len(results)} tests passed")
```

3. Test table – expected and actual results

python test_milkcalc.py

```
Test case                              Expected    Actual  Result
N1 fat+ snf+  8.5 L 4.2/8.7             283.90    283.90  PASS
N2 fat- snf-  12 L 3.1/8.3              333.60    333.60  PASS
N3 fat+ snf-  10 L 4.0/8.2              311.00    311.00  PASS
N4 fat- snf+  5 L 3.0/8.8               144.50    144.50  PASS
B1 standard   10 L 3.5/8.5              300.00    300.00  PASS
B2 lowest fat 10 L 2.0/8.5              240.00    240.00  PASS
B3 highest fat 10 L 7.0/8.5             440.00    440.00  PASS
B4 smallest qty 0.1 L 3.5/8.5             3.00      3.00  PASS
B5 half-paise 0.05 L 3.6/8.4              1.51      1.51  PASS
I1 text litres 'eight'                rejected  rejected  PASS
I2 negative litres '-5'               rejected  rejected  PASS
I3 zero litres '0'                    rejected  rejected  PASS
I4 fat above range 7.1                rejected  rejected  PASS
I5 fat below range 1.9                rejected  rejected  PASS
I6 comma decimal '8,5'                rejected  rejected  PASS
I7 'nan' as fat                       rejected  rejected  PASS
I8 snf above range 11                 rejected  rejected  PASS
H1 five-entry total (by hand)          1373.00   1373.00  PASS

18 of 18 tests passed
```

Group	Cases	What it shows
Normal (N1–N4)	fat/SNF above and below the standard in all four combinations	The step rule works in both directions (283.90, 333.60, 311.00, 144.50).
Boundary (B1–B5)	exactly 3.5/8.5; lowest fat 2.0; highest fat 7.0; smallest quantity; a half-paisa tie	Limits accepted; 0.05 L × Rs 30.10 = 1.505 is paid as 1.51 (round half up).
Invalid (I1–I8)	text, negative, zero, fat above/below the range, comma decimal, nan, SNF out of range	Every invalid entry is rejected with ValueError, none is paid.
Hand total (H1)	5 entries: 300.00 + 283.90 + 333.60 + 311.00 + 144.50	Calculated by hand as 1,373.00; program gives 1373.00.

Alternative Concept and Complexity Analysis

Alternative: test by typing inputs by hand each time and reading the answer on the screen.

	Manual typing	assert-based test function
Process	type litres, fat and SNF for each case, compare with the hand answer by eye, repeat after every change	write each case once with its expected value; run one command; a failing assert stops with the case name
Time to run all 18 cases	≈ 18 × 30 s ≈ 9 minutes (at about 30 s per case), i.e. O(m) human time for m cases, repeated after every change	≈ 22 ms including starting Python; O(m) machine time with a tiny constant
Reliability	a tired tester skips or misreads cases	identical every time; also records the expected value permanently
Rerun after each of c changes	≈ 9 min × c	≈ 0.02 s × c

Both grow linearly with the number of cases m, but the constant differs by a factor of several thousand, so only automated tests are practical when the code is changed repeatedly (rate revisions, new rules).

Evidence

Working source code and a test table with expected and actual results for every case (18 of 18 passed). Source: Task09/milkcalc.py, test_milkcalc.py.

Task 10 – Compare Expectations with Execution

Python Programming • Assignment 1 • Village Milk Co-operative Payment Calculator | Marks: 4 | Bloom's level: K4 – Analyse

Name: **Dhaarani D** Register No.: _____ GitHub: _____ (folder Task10)

1. What I expected

Before running: (1) float and Decimal totals would agree to the paisa for 300 payments and differ by a few paise at 6,000 and 150,000 because float errors accumulate; (2) Decimal would be slower; (3) rounding each payment or rounding only the total would give about the same answer.

2. Experiment

compare.py generates n = 300, 6,000 and 150,000 random entries with a fixed seed (litres with two decimals, fat 2.5–6.5, SNF 7.8–9.2) and computes the ten-day total in three ways: **float** (milkrate.py from Task 7, each payment rounded with round()), **Decimal, each payment rounded to the paisa** (milkcalc.py from Task 9) and **Decimal, only the total rounded**. Times use time.perf_counter().

compare.py (excerpt)

```
import random, time
from decimal import Decimal as D
import milkrate as fl        # float version (Task 7)
import milkcalc as dc        # Decimal version (Task 9)

def make_entries(n, seed=2025):
    rnd = random.Random(seed)
    out = []
    for _ in range(n):
        litres = f"{rnd.uniform(2, 25):.2f}"
        fat = f"{rnd.randint(25, 65) / 10:.1f}"
        snf = f"{rnd.randint(78, 92) / 10:.1f}"
        out.append((litres, fat, snf))
    return out

def float_total(entries):
    total = 0.0
    for l, f, s in entries:
        total += fl.payout(float(l), float(f), float(s))
    return total

def decimal_total(entries):
    return dc.ten_day_total([(D(l), D(f), D(s)) for l, f, s in entries])

def decimal_round_only_total(entries):
    t = D(0)
    for l, f, s in entries:
        t += D(l) * dc.rate_per_litre(D(f), D(s))     # no per-payment rounding
    return dc.to_money(t)
```

Results

```
       n          float total    Decimal total     difference  round-total only gap (paise)
     300    143587.8100000001        143587.96  -0.1499999999         143587.87           -9
         float time     0.44 ms   Decimal time     1.10 ms
    6000   2772380.9299999960       2772383.76  -2.8300000040        2772380.75         -301
         float time     8.74 ms   Decimal time    20.52 ms
  150000  69002202.1399991661      69002275.38 -73.2400008300       69002203.24        -7214
         float time   267.30 ms   Decimal time   524.34 ms
```

n	float total	Decimal total (bank)	float − Decimal	round only total	float time	Decimal time
300	143,587.81	143,587.96	−0.15	143,587.87 (−0.09)	0.44 ms	1.10 ms
6,000	2,772,380.93	2,772,383.76	−2.83	2,772,380.75 (−3.01)	8.7 ms	20.5 ms
150,000	69,002,202.14	69,002,275.38	−73.24	69,002,203.24 (−72.14)	267 ms	524 ms

3. Reasons for the difference – a second experiment

The gaps are much bigger than float accumulation error alone would give, so I separated the causes:

Same 150,000 entries with litres to 1 decimal and to 2 decimals

```
litres with 1 decimal(s): float total 69002544.220001  Decimal total 69002544.22  gap 0.0000  entries nee
litres with 2 decimal(s): float total 69002202.139999  Decimal total 69002275.38  gap -73.2400  entries n
```

- **Accumulation error is tiny:** with litres to one decimal every amount is already an exact number of paise; float and Decimal totals agree to the paisa (gap 0.0000, only 0.000001 of float noise on 69 million rupees).
- **The real cause is rounding of half-paise amounts.** With two-decimal litres an amount can fall exactly half-way between two paise, e.g. 0.15 L × Rs 30.10 = 4.515. Decimal rounds the tie up (Rs 4.52), as a bank does; with float, round(4.515, 2) gives 4.51 because the binary value is stored just below the tie, so one paisa less is paid. 7,324 of 150,000 payments differ by exactly one paisa: 7,324 × 0.01 = Rs 73.24, which is exactly the gap in the table.
- **Rounding only the total is wrong for the bank:** a bank statement is a sum of whole-paise credits. Not rounding each payment lets half-paise remainders vanish (or accumulate) and the total is Rs 72.14 away at 150,000 – a result no farmer was actually paid. **Rounding each payment, with an explicit half-up rule on exact decimals, gives the correct bank amount.**
- **Time:** both grow in proportion to n (300 → 6,000 is 20× and the time grows about 20×; 6,000 → 150,000 is 25× and time about 30×). Decimal is about twice as slow (≈ 3.5 µs against 1.8 µs per payment) but 0.5 s for 150,000 payments is acceptable for a job run once every ten days.

My first expectation was therefore partly wrong: float and Decimal already disagree at 300 payments, and for a reason (tie rounding) that is different from the one I predicted (accumulated error).

Alternative Concept and Complexity Analysis

Alternative: compute the rate for each distinct (fat, snf) pair once, keep it in a dictionary `RATE_TABLE[(fat, snf)]` and look it up for every entry.

ratetable.py (excerpt)

```
N = 150000
fats = [round(rnd.randint(25, 65) / 10, 1) for _ in range(N)]
snfs = [round(rnd.randint(78, 92) / 10, 1) for _ in range(N)]
pairs = list(zip(fats, snfs))

t0 = time.perf_counter()
for f, s in pairs:
    fl.rate_per_litre(f, s)
t1 = time.perf_counter()
table = {}
for f in range(25, 66):
    for s in range(78, 93):
        table[(f / 10, s / 10)] = fl.rate_per_litre(f / 10, s / 10)
t2 = time.perf_counter()
for p in pairs:
    table[p]
t3 = time.perf_counter()
print(f"float recalculation : {1e9*(t1-t0)/N:7.0f} ns/entry")
print(f"build table ({len(table)} pairs) : {1e6*(t2-t1):7.0f} us once")
print(f"float table lookup  : {1e9*(t3-t2)/N:7.0f} ns/entry")
```

Measured (ratetable.py)

```
float recalculation :     886 ns/entry
build table (615 pairs) :    2034 us once
float table lookup  :     187 ns/entry
Decimal recalculation:    2408 ns/entry
Decimal table lookup :     184 ns/entry  (table of 615 pairs built once)
dict size (bytes) for 615 pairs: 18512
```

	Recalculate for every entry	Rate table (dictionary)
Time per entry	float ≈ 886 ns; Decimal ≈ 2,408 ns – O(1) but with many operations	≈ 185 ns for both – O(1) hash lookup (4.7× faster for float, 13× for Decimal)
Extra space	O(1)	O(k) for k distinct pairs: 615 pairs ≈ 18.5 KB (at most 51 × 41 = 2,091 pairs for the whole allowed range)
One-time cost	none	building 615 float pairs ≈ 2 ms
Break-even	—	2,034 µs / (886 − 187) ns ≈ 2,900 entries; a single centre has about 300 × 20 = 6,000 entries per cycle, so it already pays off

For n entries: recalculation costs O(n) with a large constant; the table costs O(k + n) with a tiny constant, where k ≤ 2,091 is independent of n. Because k is fixed and small while n grows to millions, the table is the better design for the district union – provided it is rebuilt whenever the rate card changes (a stale table would reintroduce the “partly applied revision” bug).

Evidence

Measured totals and times above for 300, 6,000 and 150,000 payments, with the reason for every difference. Source: Task10/compare.py, ratetable.py.

Task 11 – Recommend and Defend the Final Design

Python Programming • Assignment 1 • Village Milk Co-operative Payment Calculator | Marks: 4 | Bloom's level: K5 – Evaluate

Name: **Dhaarani D** Register No.: _____ GitHub: _____ (folder Task11)

1. Recommended design

Decision	Recommendation	Evidence from my experiments
Type of money	**Decimal** built from the text the clerk typed (or integer paise); never float	Task 2: float gave 33400.30000000061 and Decimal(float(...)) kept the error; Task 10: 7,324 wrong paise (Rs 73.24) in 150,000 float payments.
Rounding rule	Round **each payment** to the paisa, **half up**, with Decimal; add the rounded amounts	Task 10: rounding only the total was Rs 72.14 away from the sum of paid amounts; Task 9 test B5 (1.505 → 1.51).
Step counting	round() to the nearest step on exact Decimal values	Task 4: int() lost a step for fat 3.1 and both int() and floor() lost a step for fat 4.1 (Rs 0.40/L each); round() was right in every case.
Structure	Constants in one rate card; small functions (rate_per_litre, payout, ten_day_total) in a module; validation before calculation	Task 3: one edit per revision; Task 6: function overhead ≈ 10 %, negligible; Task 9: 18 of 18 tests pass.
Speed-up	Optional rate table of (fat, snf) pairs	Task 10: 886 → 187 ns per entry (float), 2,408 → 184 ns (Decimal).
Memory at scale	Running total per farmer; do not keep all entries unless needed	Task 8: O(1) versus 16.5 MB at n = 150,000.

2. How a rate revision will be carried out

- Edit only the rate card (BASE_RATE, FAT_STEP_VALUE, SNF_STEP_VALUE) – one place, no other file.
- Run test_milkcalc.py; the expected values of the rate-dependent tests are recalculated by hand and updated at the same time. All other tests (validation, rounding, hand total) must still pass.
- Commit the change with the date and the union's order number; run one old entry through both old and new rate to confirm the difference equals the revision.
- If a rate table (Task 10) is used, rebuild it at start-up so it can never be stale.

3. Limitations of the design

- Readings are assumed to have one decimal; the two-decimal analyser needs the pro-rata step rule (Task 12).
- Only one rate card at a time and no bonus rules; seasonal rates need a rate card per season (Task 12).
- SNF limits (6.0–10.0) and the 200 L entry cap are my assumptions and must be confirmed with the union.
- Decimal is about two to three times slower than float (Task 10) and needs the rule "never build Decimal from float" to be followed by every programmer.
- A text rate card must be validated; a typing slip in the file (0.04 for 0.40) would be paid silently unless range checks exist.
- The program calculates payments but does not store them; a database and audit trail are outside this assignment.

4. Is the design correct at district-union scale?

Correctness: yes – Decimal arithmetic is exact at any n, the rounding rule is defined per payment, and the 18 tests do not depend on n. **Speed:** measured ≈ 3.5 µs per payment with Decimal. A district union has 150,000 farmers; at an assumed 20 entries per farmer per ten days that is 3 million

entries, i.e. roughly 10 seconds (an estimate from my measurement) – acceptable for a job run every ten days, and the rate table reduces it further. **Memory:** O(1) with running totals; O(n) only if all entries are kept (16.5 MB per 150,000 entries measured).

Alternative Concept and Complexity Analysis

Alternative: read the rate card from a small JSON file when the program starts, instead of writing it in the module.

ratecard.json and load_card.py (excerpt)

```
{
  "base_rate": "30.00",
  "fat_step_value": "0.40",
  "snf_step_value": "0.30",
  "std_fat": "3.5",
  "std_snf": "8.5",
  "step": "0.1"
}

import json, time, timeit
from decimal import Decimal

def load_card(path="ratecard.json"):
    with open(path) as fh:
        raw = json.load(fh)
    required = {"base_rate", "fat_step_value", "snf_step_value", "std_fat", "std_snf", "step"}
    missing = required - raw.keys()
    if missing:
        raise ValueError(f"rate card is missing: {sorted(missing)}")
    return {k: Decimal(v) for k, v in raw.items()}
```

Measured (load_card.py)

```
{'base_rate': '30.00', 'fat_step_value': '0.40', 'snf_step_value': '0.30', 'std_fat': '3.5', 'std_snf': '8.5', 'step':
start-up cost to read 6 values: 20 us
memory of the card (dict + values): 896 bytes
```

	Rate card in the module	Rate card in a JSON file
Start-up time (t = 6 values)	none (values compiled in)	≈ 20 µs: O(t) to read and parse; irrelevant beside Python start-up
Space	t constants	O(t): about 900 bytes for t = 6 Decimal values
After a revision	edit the .py file, retest, redistribute the program to every collection centre	edit one small text file; the program is not touched or redistributed
Risk	a code edit may break something else	a bad value or missing key; mitigated by validation (the loader checks the required keys)

Reading six values costs about 20 microseconds once per run – negligible – while redistributing code to every centre after each revision costs people time and risks version mismatch. The JSON file is therefore recommended *for the values that change* (rate card, season dates), with the validation shown, and the calculation code stays in the module.

Evidence

A recommendation supported by the measurements of Tasks 2, 4, 6, 8, 9 and 10 and the JSON loading test. Source: Task11/ratecard.json, load_card.py.

Task 12 – Re-evaluate and Adapt the Design (Day-4 Design Challenge)

Python Programming • Assignment 1 • Village Milk Co-operative Payment Calculator | Marks: 8 | Bloom's level: K5 – Evaluate

Name: **Dhaarani D** Register No.: _____ GitHub: _____ (folder Task12)

Decision: MODIFY. The module/function structure, Decimal money and the tests are retained; the step rule, the constants (a rate card per season) and the ten-day statement (a new bonus function) are changed. A full redesign is not needed.

A. Impact Analysis (2 marks)

What has changed	Affected part of the original design
Analyser sends fat and SNF with two decimals (e.g. 4.24)	**Expression** for steps: round((fat − 3.5)/0.1) now throws away up to half a step (Rs 0.20/L). **Rounding**: tie readings such as 3.55 occur often. Input validation already accepts decimals as text.
Seasonal rates: April–June base Rs 32.00 and fat step Rs 0.45	**Constants** BASE_RATE and FAT_STEP_VALUE can no longer be single values; **functions** rate_per_litre and payout need to know the season. SNF step unchanged at Rs 0.30.
Bonus Rs 0.50 per litre if more than 500 litres in ten days	A **new function** and a **new constant pair**; the bonus depends on the ten-day litres, so it belongs in the ten-day statement, not in payout() for one entry.

How two-decimal readings change step calculation and rounding

A reading of 4.24 is 7.4 steps above 3.5. The original rule rounds this to 7 steps; with two-decimal readings the farmer's extra 0.04 % of fat is ignored, while 4.26 would be rounded up to 8 and overpay. The fair rule is **pro rata**: use the fractional step count (7.4) and round only the final rate or amount to the paisa. For fat 4.24 and SNF 8.58 (normal season) the original design gives Rs 33.10 per litre, the pro-rata rule Rs 33.20; on a 500 L delivery that is Rs 50.00 difference.

B. Evaluation of the Original Design (2 marks)

Part of original design	Still appropriate?	Worked example
Decimal money, ROUND_HALF_UP to paise, validation, tests	**Yes – retain**	No change needed: Decimal("4.24") parses exactly; payments are still whole paise.
Module with small functions	**Yes – retain**	Only the bodies of rate_per_litre and payout change; callers keep the same shape.
Single BASE_RATE / FAT_STEP_VALUE constants	**No – modify**	Summer 3.5/8.5 must be Rs 32.00, normal Rs 30.00: 10 L → 320.00 vs 300.00.
round() to a whole step	**No – modify**	fat 4.24 / SNF 8.58 → Rs 33.10 (old) vs Rs 33.20 (fair). Tie readings (3.55, 3.65, 3.75) give 0, 1, 2 steps with float round() (representation error and banker's rounding) but 1, 2, 3 with half-up; pro-rata steps avoid the problem.
payout() for one entry	**Partly**	Keep, but add season; bonus cannot be computed here because it needs all litres of the ten days.

C. Adaptation – revised design and working code (2 marks)

- **Rate cards in one dictionary** RATE_CARDS = {"normal": …, "summer": …}; a revision or a new season is a new entry.
- **season_for_month(month)** chooses the card (April = 4 to June = 6).
- **Season passed as a parameter** to rate_per_litre, payout and ten_day_statement (default "normal", so old calls still work).
- **Pro-rata steps** on Decimal: (fat − 3.5)/0.1 without rounding; money rounded once per payment.

- **Separate bonus(total_litres)** function: Rs 0.50 on all litres when the ten-day total exceeds 500 L (assumption: all litres, not only the excess; changing the rule affects one line).

milkseason.py

```
"""Season-aware payment calculator (changed requirement)."""
from decimal import Decimal as D, ROUND_HALF_UP

PAISA = D("0.01")
STD_FAT, STD_SNF, STEP = D("3.5"), D("8.5"), D("0.1")
SNF_STEP_VALUE = D("0.30")

RATE_CARDS = {                      # one entry per season: the ONLY place rates live
    "normal":   {"base": D("30.00"), "fat_step": D("0.40")},
    "summer":   {"base": D("32.00"), "fat_step": D("0.45")},    # April - June
}
BONUS_LITRES = D("500")             # more than this in ten days earns the bonus
BONUS_PER_LITRE = D("0.50")


def money(x):
    """Round to whole paise, half up."""
    return x.quantize(PAISA, rounding=ROUND_HALF_UP)


def season_for_month(month):
    """April (4) to June (6) is the summer season."""
    return "summer" if 4 <= month <= 6 else "normal"


def rate_per_litre(fat, snf, season="normal"):
    """Rate per litre; readings may have two decimals, so steps are fractional (pro rata)."""
    card = RATE_CARDS[season]
    fat_steps = (fat - STD_FAT) / STEP         # e.g. 4.23 -> 7.3 steps, not rounded to 7
    snf_steps = (snf - STD_SNF) / STEP
    return card["base"] + fat_steps * card["fat_step"] + snf_steps * SNF_STEP_VALUE


def payout(litres, fat, snf, season="normal"):
    """Amount for one delivery, before bonus, rounded to paise."""
    return money(litres * rate_per_litre(fat, snf, season))


def bonus(total_litres):
    """Bonus for ten days: Rs 0.50 on ALL litres if more than 500 litres were supplied."""
    return money(total_litres * BONUS_PER_LITRE) if total_litres > BONUS_LITRES else D("0.00")


def ten_day_statement(entries, season="normal"):
    """entries = list of (litres, fat, snf); returns (litres, milk amount, bonus, total)."""
    litres = sum((l for l, _, _ in entries), D("0"))
    milk = sum((payout(l, f, s, season) for l, f, s in entries), D("0.00"))
    b = bonus(litres)
    return litres, milk, b, milk + b
```

run_season.py

```
A. same reading, both seasons (10 L, fat 4.2, snf 8.7)
   normal  rate  33.40  pay   334.00
   summer  rate  35.75  pay   357.50
B. two-decimal analyser readings (10 L, fat 4.24, snf 8.58, normal season)
   old design (round to whole step): rate 33.10
   new design (pro rata steps)      : rate 33.20
   difference on a 500 L delivery   : Rs 50.00
C. bonus boundary (litres in ten days, all at 3.5/8.5, summer)
    499.9 L  milk   15996.80  bonus    0.00  total   15996.80
      500 L  milk   16000.00  bonus    0.00  total   16000.00
    500.1 L  milk   16003.20  bonus  250.05  total   16253.25
D. season from month
   month 3: normal
   month 4: summer
   month 6: summer
   month 7: normal
```

D. Evidence-Based Defence (2 marks)

test_season.py – tests for both seasons, two-decimal readings and the bonus

```
Test                                Expected      Actual  Result
normal 10 L 3.5/8.5                   300.00      300.00  PASS
summer 10 L 3.5/8.5                   320.00      320.00  PASS
summer 10 L 4.2/8.7                   357.50      357.50  PASS
two-decimal 10 L 4.24/8.58 nrm        332.00      332.00  PASS
bonus at exactly 500 L                  0.00        0.00  PASS
bonus at 500.1 L                      250.05      250.05  PASS
bonus at 600 L                        300.00      300.00  PASS
statement 2 x 300 L summer          19500.00    19500.00  PASS
April is summer                       summer      summer  PASS
July is normal                        normal      normal  PASS
10 of 10 passed
```

Evidence	What it shows
Both seasons (tests 1–3)	Normal 3.5/8.5 = 300.00 and summer = 320.00 for 10 L; summer 4.2/8.7 = Rs 35.75/L × 10 = 357.50 (= 32.00 + 7×0.45 + 2×0.30).
Two-decimal reading (test 4)	4.24/8.58 gives Rs 33.20/L = 332.00 for 10 L, hand-calculated: 30 + 7.4×0.40 + 0.8×0.30. The old design would pay Rs 33.10.
Bonus (tests 5–8)	Exactly 500 L: no bonus (“more than 500”); 500.1 L: Rs 250.05; 600 L: Rs 300.00; two 300 L deliveries in summer: 19,200.00 + 300.00 = 19,500.00.
Season selection (tests 9–10)	April and June give summer, July gives normal.

Trade-offs. *Gain:* the farmer is paid exactly for the quality measured, every season’s rates live in one table, the old tests carry over and 10 of 10 new tests pass. *Cost:* a more complex rate function and one more parameter everywhere; pro-rata steps make amounts depend on fractional steps, so the union must confirm that it wants pro-rata rather than rounding to 0.1 % steps. *Not chosen:* a full redesign (a class or database) would add code and risk with no gain for this change; keeping the original unchanged would underpay or overpay by up to Rs 0.35 per litre (0.20 from fat plus 0.15 from SNF) – Rs 175 on a 500 L delivery – and could not express the season or the bonus. *Assumptions to confirm:* the bonus applies to all litres, and the SNF step stays Rs 0.30 in every season.

Evidence

Impact analysis, evaluation with worked examples, a working modified design and test results for both seasons and the bonus. Source: Task12/milkseason.py, run_season.py, test_season.py.